Autonomous Knowledge Engineering System

Design Decisions, Architecture, and Lessons from Industry

A blueprint for continuous, agent-driven organizational knowledge

Architecture & Design Brief
Prepared June 2026

Contents

1. The Problem: Why Organizational Knowledge Decays

Every engineering organization accumulates two parallel records of how its systems work. The first is the system itself — code, infrastructure-as-code, pipelines, and runtime behavior. The second is the *described* system — architecture diagrams, runbooks, wikis, ADRs, Slack threads, and tribal knowledge held by individuals. These two records start aligned and diverge continuously from the moment they are created.

This divergence is not a tooling failure; it is a structural property of how software organizations work. Documentation is written at a point in time and read at a different point in time, after the system has changed. The cost of updating documentation is paid by the person making a code change, but the benefit accrues to someone else, later, often during an incident. This is a classic externality problem, and it explains why documentation drift is universal regardless of team discipline or tooling maturity.

The consequence shows up most acutely in three moments:

- **Onboarding** — new engineers spend weeks reconstructing a mental model of the system from a combination of outdated docs, source code, and asking people who may have moved teams.
- **Incidents** — on-call engineers at 3am need to know which services depend on which, and existing dashboards, logs, and wikis each show a fragment of the picture.
- **Architectural decisions** — new design proposals are made without full awareness of prior decisions, constraints, or the reasons earlier approaches were rejected, because that context lives in closed Slack threads or the memory of someone on parental leave.

The "Autonomous Knowledge Engineering System" described in this brief is not a new idea in isolation — it is the logical extension of a a path that Google, Spotify, Netflix, Uber, and others have each walked independently over the last fifteen years, driven by the same underlying pressure: **engineering organizations scale faster than human-maintained documentation can keep up**. What is new is that large language model agents can now plausibly close the loop — reading code, reading conversations, reasoning about intent, and writing documentation back — continuously, rather than as a quarterly initiative.

Framing for this document. Section 2 looks at how tech giants solved adjacent problems — service catalogs, code intelligence, dependency mapping — and what each solution optimized for. Section 3 reasons from those case studies to a general architecture. Sections 4 onward present that architecture, the design decisions behind it, and a path to build it.

2. How Tech Giants Have Approached This

No company built "an autonomous knowledge engineering system" as a single product. Instead, each company solved a narrower, painful problem first, and the resulting systems became the substrate that later knowledge tooling was built on. The pattern across all of them is: **start with a structural index of what exists (catalog or code graph), then layer relationships, then layer documentation, then (now) layer AI agents on top.**

Spotify — Backstage: the Service Catalog as the Front Door

By the mid-2010s Spotify's engineering organization had grown into hundreds of autonomous squads, each free to choose their own tools and practices. The benefit was speed; the cost was that infrastructure became fragmented and engineers became less productive because nobody had a single place to find out what existed, who owned it, or how it worked.

Spotify's answer was a software catalog: an internal service catalog meant to solve data silos, internal inconsistencies, and lack of integration, automation, and overview. The catalog was built around a simple idea — metadata YAML files stored alongside the code, harvested and visualized centrally — which meant ownership data lived in the same git workflow developers already used, rather than in a separate system that would inevitably go stale. The catalog started simple, then TechDocs (documentation) and Software Templates were added, reaching the first internal version by 2018, before being open-sourced as Backstage in 2020 and donated to the CNCF.

The core lesson: **ownership and discovery had to be solved before documentation quality could be solved**, because you cannot maintain documentation for things you cannot enumerate or attribute to a team. Backstage's catalog is essentially a lightweight knowledge graph (component → owner → API → docs → dependencies) maintained via the existing developer workflow rather than a separate curation effort.

Sources: Spotify/Backstage documentation (backstage.io, backstage.spotify.com)

Google — Code Search, Kythe, and the Code Graph

Google's problem was different in kind: a single monorepo too large for any individual IDE-based indexing approach to scale. If every developer's IDE did its own indexing, the work would scale roughly quadratically — the codebase grows linearly while a linearly growing number of IDEs each do more work — which is not a recipe for good scaling.

The solution was Kythe (originally an internal project called Grok): Kythe instruments the build workflow to extract semantic nodes and edges from source code, collecting cross-reference graphs for each build rule, which are then merged into one global graph optimized for common queries like go-to-definition and find-all-usages. This produced a language-agnostic code graph that Code Search, IDE plugins, review tools, and (now) AI migration agents all query.

The payoff became visible recently in AI-driven code migrations. An expert engineer identifies a "seed" — for example a protocol buffer field to migrate — then uses Kythe to find direct references and repeatedly searches for references-to-references in a breadth-first fashion, producing a superset of locations that might need modification. Each candidate location is then categorized by confidence — not-yet-migrated, irrelevant, relevant, or left-over for manual review — before an LLM proposes changes that are validated against unit tests. The result: AI wrote nearly 75% of the code in this migration and developers reported a 50% productivity increase, on a task that previously took roughly two years to complete manually.

The lesson: **a structural graph of "what calls what" / "what depends on what" is the prerequisite for AI agents to operate safely at scale** — it bounds the blast radius of any given change or claim, and it lets confidence scoring be grounded in graph distance and automated checks rather than vibes.

Sources: Google Research blog, abseil.io (Software Engineering at Google), arXiv 2501.06972

Netflix — Service Topology: Real-Time Dependency Mapping from Multiple Sources

Netflix's microservices architecture grew to the point where, when a member presses play, that single action triggers a cascade of service-to-service calls spanning authentication, recommendations, video encoding selection, and playback optimization. Traditional observability tools each showed fragments of this picture — metrics show symptoms, logs show individual service behavior — leaving an engineer at 3am to mentally stitch together multiple tools, which is slow, error-prone, and stressful.

Netflix's response, Service Topology, is notable for how it sources truth: it ingests data from three sources stored in separate graph partitions — eBPF network flow logs that capture kernel-level traffic regardless of instrumentation, IPC

metrics from instrumented services that add endpoint and protocol detail, and a third source — merging them into a single, near-real-time queryable graph. Critically, the team learned that no single source of dependency information tells the complete story — network data lacks application context, and application metrics only cover instrumented services — so multiple perspectives had to be combined, and data quality was treated as critical because incomplete or incorrect dependency information is worse than no information during an incident.

The lesson: **"living" architecture documentation cannot be derived from a single source** (not just code, not just runtime traces, not just docs) — it must reconcile multiple, sometimes-contradictory signals, and the system must be explicit about its own confidence rather than presenting a single authoritative-looking diagram.

Sources: Netflix Tech Blog, InfoQ coverage of Netflix Service Topology and Edgar

Netflix / LinkedIn / Uber — Graph-Based Metadata Platforms Beyond Code

The graph pattern has extended beyond service dependencies into data and ML systems. Netflix's Model Lifecycle Graph maps interconnections between datasets, models, features, and workflows to improve discoverability, governance, and component reuse, and this mirrors a broader industry movement toward metadata-centric platforms, including LinkedIn DataHub (which models relationships between datasets, pipelines, and ownership as a graph) and Uber's Michelangelo platform, which emphasized centralized lifecycle management and reproducibility. The same graph-based representation pattern — modeling relationships between services, infrastructure, ownership, and operational metadata — increasingly appears across internal developer portals such as Spotify's Backstage as well.

The lesson: by 2026, the "knowledge graph as substrate" pattern has converged across data platforms, ML platforms, and developer platforms independently — strong evidence that it is the right general-purpose answer rather than a one-off solution.

Sources: InfoQ coverage of Netflix Model Lifecycle Graph

Cross-Cutting Pattern Across Netflix, Uber, Amazon, and Spotify

Looking across these companies' microservice journeys together, a consistent set of structural investments recurs: platform engineering providing shared infrastructure that abstracts away complexity, unified observability so distributed systems can actually be monitored and debugged, clearly defined ownership with accountability written into team structures, and resilience designed in from the start through fault-tolerant patterns and chaos testing. Spotify in particular combined two complementary mechanisms: "golden paths" — curated tools and practices teams could adopt without being forced to — and Backstage as the centralizing developer portal, which together reduced the cost of team autonomy while improving consistency.

The lesson for an autonomous knowledge system: **it works best as connective tissue across platforms that already enforce some structure** (ownership metadata, golden paths, standard CI/CD) — it amplifies existing discipline rather than substituting for it.

Sources: Netguru industry analysis of Netflix/Uber/Amazon/Spotify microservice scaling

What's New Now (2025–2026): The Closing of the Loop

The 2026 Netflix Service Topology launch and the 2025 Google AI migration results share a common thread: both are recent (within the last year), and both show the same systems that were originally built for *human consumption* (dependency graphs, code graphs) becoming the substrate that *AI agents* now query and act on. Spotify's Backstage has followed the same arc — its 2026 changelog shows new MCP actions that let AI agents such as Claude Code and IDE extensions query and manage the platform's compliance system (Soundcheck) conversationally, asking about failing checks and reviewing compliance through natural language.

This is the crucial reasoning step for the Autonomous Knowledge Engineering System: **the graphs and catalogs already exist at these companies** — what changed in the last 18 months is that they became queryable and actionable by AI agents via standard protocols (MCP), turning static reference systems into a live substrate for autonomous documentation work.

3. Reasoning: From Case Studies to a General Architecture

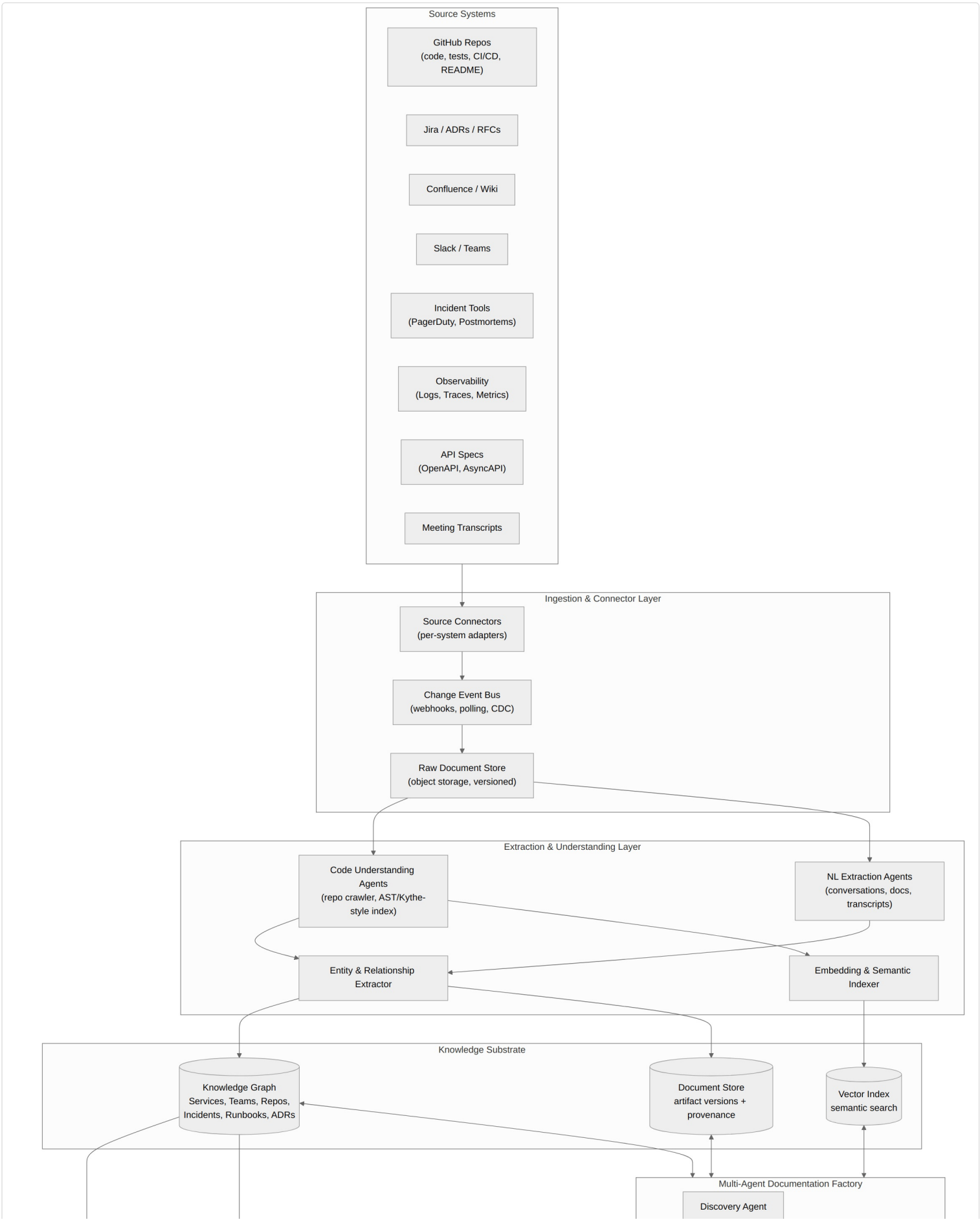
Each case study above was a point solution to a specific scaling pain. Generalizing across them produces a small set of design principles that drive the architecture in Section 4:

Observation from industry	Generalized principle
Spotify could not maintain documentation until it could enumerate ownership (Backstage catalog)	**Discovery and ownership attribution come first.** A knowledge system must know what exists and who is accountable before it can usefully document anything.
Google's Kythe graph turns "find all usages" from an IDE-local quadratic problem into a centrally-computed, queryable graph	**Build the structural graph once, centrally, and let many consumers (humans and agents) query it.** Don't re-derive structure per-query.
Google's migration agents bound their changes using graph distance and confidence buckets (not-migrated / irrelevant / relevant / left-over)	**Confidence scoring and blast-radius bounding are first-class.** Agents should reason in terms of "how far is this claim from verified ground truth" not produce flat true/false documentation.
Netflix found that no single source (network data, app metrics, traces) tells the complete dependency story	**Multi-source reconciliation is mandatory.** Code, runtime telemetry, and human-authored docs must all feed the graph, with explicit conflict resolution.
Netflix treats "incomplete or incorrect dependency info as worse than no info" during incidents	**Wrong-but-confident output is the primary risk to design against** — more so than missing output. Drives the governance/trust model in Section 10.
Backstage's golden paths + catalog reduced the cost of team autonomy without removing it	**The system should be additive to existing team workflows** (git, PR review, existing wikis) rather than a new system of record teams must adopt separately.
2025–2026: Kythe, Backstage, and Netflix's graphs became agent-queryable via MCP	**Treat the knowledge graph itself as an MCP-exposed resource from day one** — it is both the system's output and an input other agents will consume.

One more reasoning step is specific to this system and not present in the case studies above: **documentation drift detection is the only research area in the original brief with a built-in feedback loop and an objective ground truth** (code/runtime is truth; docs are claims to be checked against it). Every other research area — living architecture docs, tribal knowledge capture, incident-to-knowledge — produces *new* claims that need the same verification step. This makes drift detection the natural "wedge": build it first, and the confidence-scoring and validation machinery it requires becomes the shared substrate for everything else.

4. System Architecture

The architecture below follows directly from the principles in Section 3. It has five layers: source systems (the organization's existing tools, unchanged), an ingestion layer that turns heterogeneous sources into a common event stream, an extraction layer that produces structured facts from both code and natural language, a knowledge substrate (graph + vector index + versioned document store) that is the single place all facts live, and a multi-agent factory that consumes the substrate to produce and continuously maintain trusted knowledge packs.



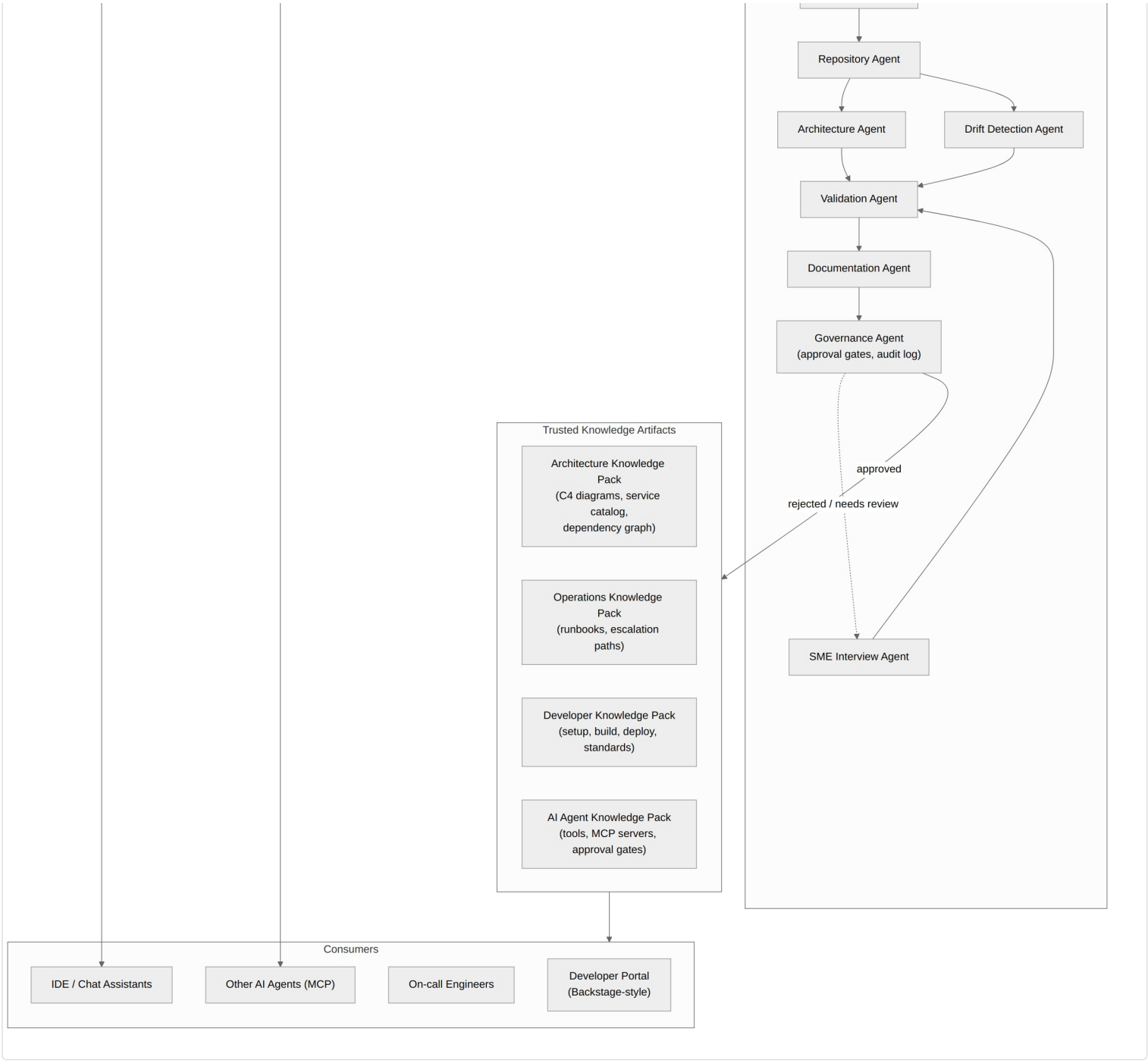


Figure 1 — End-to-end system architecture: sources, ingestion, extraction, knowledge substrate, multi-agent factory, and consumer-facing knowledge packs.

Layer-by-layer rationale

Source systems (unchanged)

Deliberately, this system does not ask teams to change where they work. GitHub, Jira, Confluence, Slack, incident tools, and observability platforms remain the systems of record for their respective domains. This mirrors Spotify's choice to keep ownership metadata in the existing git workflow rather than a separate curation tool — the knowledge system reads from these sources, it does not replace them.

Ingestion & connector layer

Per-system adapters normalize events (commits, PR merges, ticket transitions, incident creation, message posts) into a common change-event format on an event bus. A versioned raw document store preserves the original artifacts with provenance — critical for the citation and audit requirements in Section 10. This layer is intentionally "dumb": its job is reliable capture, not interpretation.

Extraction & understanding layer

Two parallel extraction paths mirror the Netflix lesson that no single source tells the complete story:

- **Code understanding agents** build a structural index analogous to Kythe — call graphs, dependency graphs, API

surfaces, infrastructure-as-code parsing — giving every claim a graph-distance-bounded scope.
- **NL extraction agents** process Slack/Teams threads, meeting transcripts, PR discussions, ADRs, and wiki pages to extract entities, decisions, and rationale that never appear in code.

Both paths feed an entity & relationship extractor that resolves references (e.g., "the payments service" in a Slack message ↔ `payments-api` repo ↔ `payments-svc` in the catalog) before writing to the graph.

Knowledge substrate

Three co-located stores, detailed in Section 5: the knowledge graph (entities and relationships), a vector index (semantic search over documents and code), and a versioned document store (every artifact version plus its provenance chain — which sources, which agent, what confidence).

Multi-agent documentation factory

Eight agent roles (Section 6) consume the substrate to discover changes, propose updates, detect drift, validate claims, draft documentation, and gate publication. This is where governance lives.

Trusted knowledge packs & consumers

Four output packs (Section 8) are generated as queries/views over the substrate rather than independently-maintained documents, so they cannot drift from each other the way today's architecture docs, runbooks, and onboarding guides drift from one another. Consumers include human-facing surfaces (developer portal, on-call tooling) and, following the Backstage/MCP precedent, other AI agents querying the graph directly.

5. The Knowledge Graph as Substrate

The four output packs requested in the brief (Architecture, Operations, Developer, AI Agent) overlap heavily — a service appears in all four, an incident appears in Operations but references services from Architecture, a runbook is referenced from both Operations and Incident records. If each pack is generated and maintained independently, they will drift from each other exactly as today's separate documents do. The resolution, following the pattern seen at Netflix and LinkedIn (Section 2), is to treat the four packs as **views over one graph** rather than four documents.

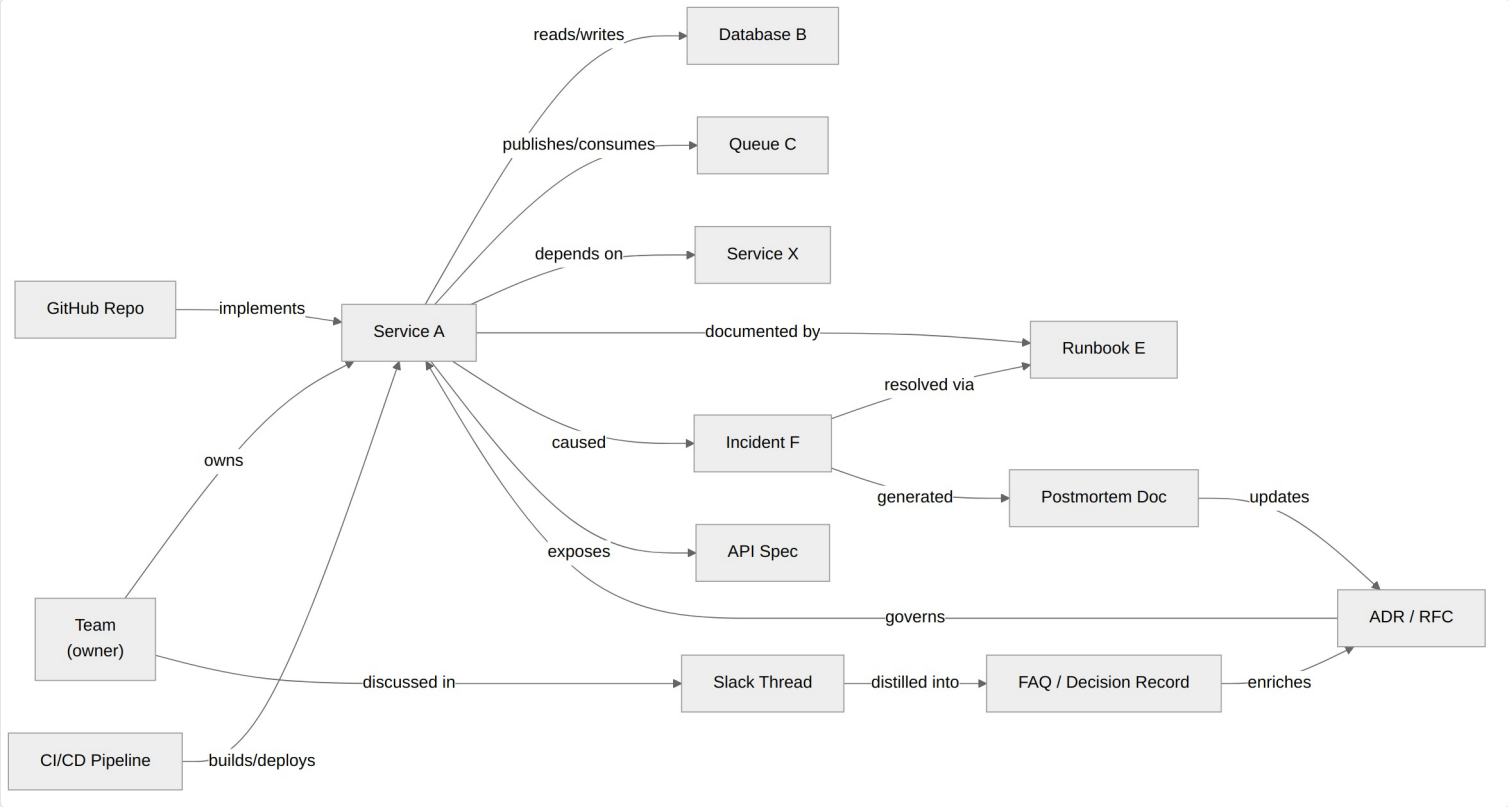


Figure 2 — Core entity types and relationships. This is illustrative; the production schema would also include API contracts, environments, and SLOs as first-class nodes.

Entity types (minimum viable schema)

Entity	Examples	Primary source(s)
Service / Component	payments-api, checkout-worker	Repo structure, Backstage-style catalog, CI/CD
Team	Payments Platform team	Catalog ownership metadata, org directory
Repository	github.com/org/payments-api	GitHub
Data store / Queue	Postgres instance, Kafka topic	IaC, code (connection strings/configs), runtime traces
API contract	OpenAPI spec for payments-api	API spec repos, code annotations
ADR / RFC	"ADR-014: Move to event-driven settlement"	ADR repo/wiki
Incident / Postmortem	INC-4471, "Checkout outage, 2026-03-02"	Incident tooling, postmortem docs
Runbook	"Payments DB failover procedure"	Runbook wiki, generated by factory
Decision/FAQ record	Distilled Slack discussion	Slack/Teams, PR discussions

Relationship types and why each matters

- **owns** (Team → Service) — without this, nothing else can be routed for review or escalation. This is the Backstage lesson: ownership is the foundation.
- **depends on** (Service → Service/DB/Queue) — derived primarily from code/IaC analysis (the Kythe-style structural graph), cross-checked against runtime traces (the Netflix Service Topology lesson).
- **documented by / resolved via** (Service/Incident → Runbook) — connects operational knowledge to the services it applies to, enabling the Operations Knowledge Pack to be generated per-service.
- **caused / generated** (Service → Incident → Postmortem) — the backbone of incident-to-knowledge automation (Section 8).

- **governs / updates** (ADR ↔ Service, Postmortem → ADR) — captures the "why" behind current architecture, including decisions later revised by incidents.
- **discussed in / distilled into** (Team → Slack Thread → FAQ → ADR) — the tribal-knowledge capture pipeline; this is the relationship type most organizations have zero structured record of today.

Provenance as a first-class property

Every node and edge carries: source artifact reference(s), extracting agent, extraction timestamp, confidence score, and verification status (verified / unverified / disputed / superseded). This is not optional metadata — it is what makes the "trusted" in "trusted knowledge artifacts" meaningful, and it is what the Validation and Governance agents (Section 6) operate on.

6. The Multi-Agent Documentation Factory

The brief proposes seven agent roles; the architecture in Section 4 consolidates these into eight roles with clearer handoffs, organized as a pipeline triggered by change events (a commit, an incident closing, a PR merging) rather than as a batch job. This event-driven framing matters: it is what makes documentation "living" rather than "periodically regenerated."

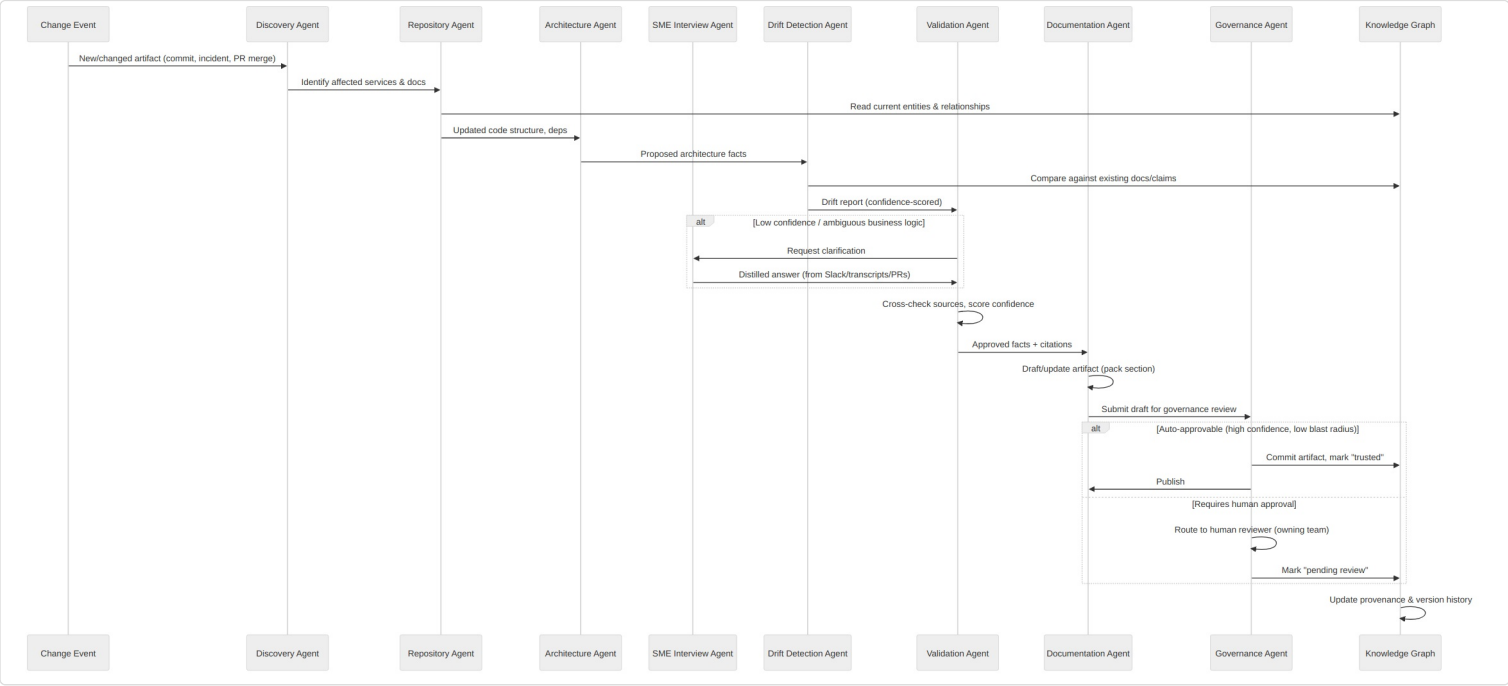


Figure 3 — Agent pipeline triggered by a change event, including the SME interview fallback and the governance approval branch.

Agent roles and responsibilities

Agent	Responsibility	Key design constraint
Discovery	Identifies which services, docs, and graph entities are affected by an incoming change event.	Must be fast and cheap — runs on every change event, so it should be mostly graph lookups, not LLM calls.
Repository	Reads current code structure, dependencies, and existing graph entries for affected services.	Source of structural "ground truth," analogous to Kythe queries.
Architecture	Derives updated architecture facts (new dependency, changed data flow, new component) from Repository Agent output.	Produces *proposed* facts only — never writes directly to "trusted" artifacts.
Drift Detection	Compares proposed/derived facts against existing documentation claims in the graph; computes drift scores.	The core of Section 7 — this agent is the system's primary trust mechanism.
SME Interview	When confidence is low or business logic intent is ambiguous, generates targeted questions for the owning team (via Slack/Teams) and distills responses back into facts.	Only invoked on demand — this is the most expensive path (human time) and must be used sparingly and well-targeted.
Validation	Cross-checks facts from multiple agents/sources, assigns final confidence scores, and decides what's ready for drafting.	Holds the confidence-scoring model; must be auditable (Section 10).
Documentation	Drafts or updates the relevant section(s) of the knowledge packs, with inline citations to source artifacts.	Produces diffs against existing artifacts, not full rewrites — preserves human edits and history.
Governance	Applies approval-gate policy (auto-publish vs. route to human reviewer), maintains the audit log, and manages "trusted" status transitions.	Policy-driven, not LLM-driven — this is the one component that should be simple, deterministic, and easy to audit.

Coordination model

Agents communicate via the knowledge graph and a shared task queue rather than direct agent-to-agent calls wherever possible — this keeps the system debuggable (every intermediate fact is inspectable in the graph with provenance) and lets agents be independently re-run, replaced, or scaled. The sequence in Figure 3 shows the conditional branches that matter most: the SME Interview escalation (only triggered on low confidence) and the Governance approval split (auto-publish for high-confidence, low-blast-radius changes vs. human review for everything else).

Conflict resolution

When sources disagree — e.g., a Slack thread says "we deprecated the legacy endpoint" but the code still serves it — the Validation Agent does not pick a winner. It records both claims with their provenance and confidence, marks the entity as **disputed**, and surfaces this explicitly in the relevant knowledge pack ("documentation claims X; code currently shows Y; unresolved as of [date]"). This is more useful than a single falsely-confident answer, directly reflecting the Netflix lesson that incomplete-or-wrong is worse than absent.

7. Documentation Drift Detection — The Wedge

As argued in Section 3, drift detection should be built first because it is the only research area with an objective ground truth and a natural feedback loop. It is also, independently, one of the highest-value capabilities on its own: simply telling a team "your runbook says the database is Postgres, the code now points at Aurora" has immediate value with no downstream packs required.

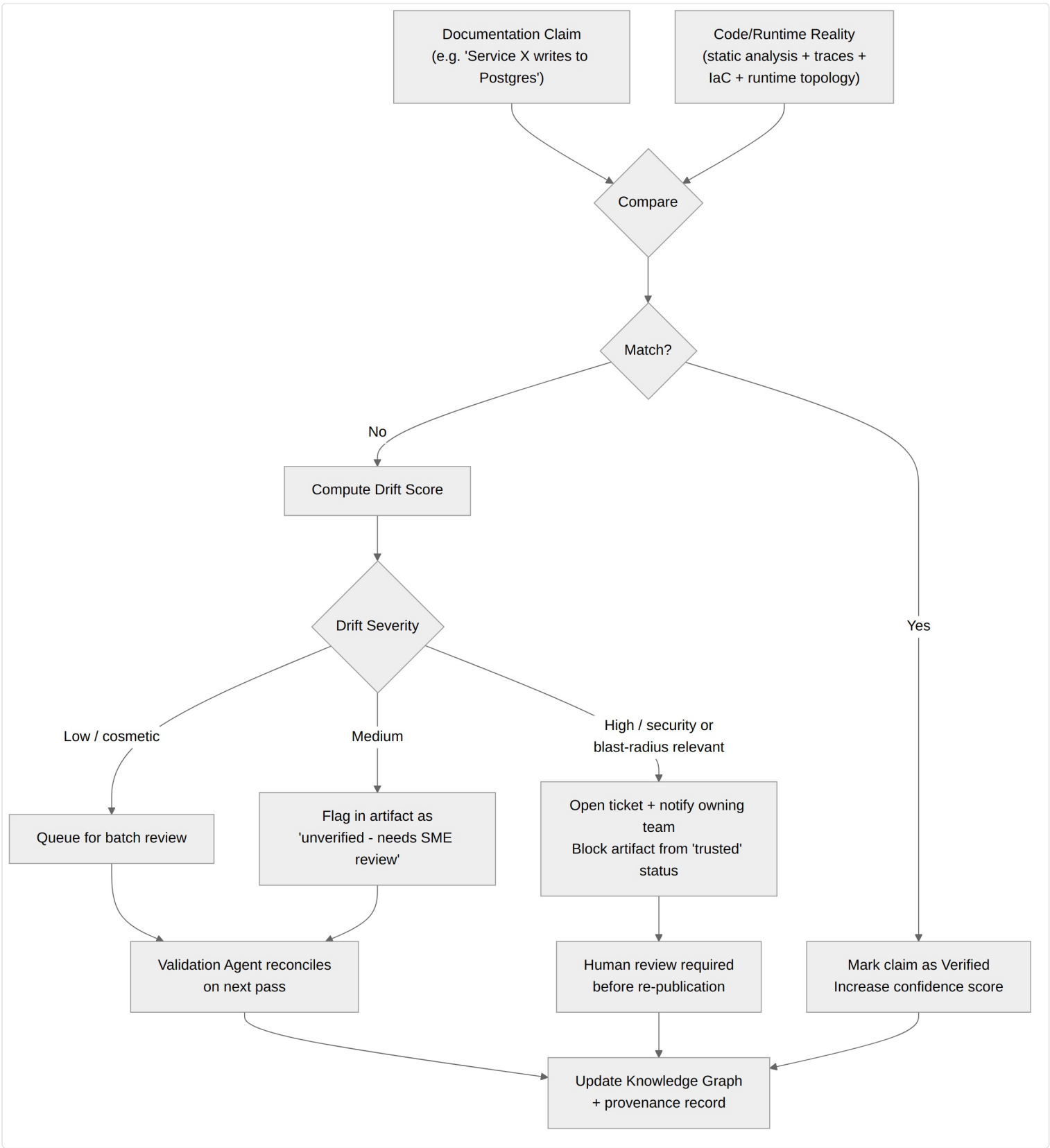


Figure 4 — Drift detection flow: every documentation claim is continuously compared against code/runtime reality, scored, and routed by severity.

Drift scoring model

Drift is not binary. A practical scoring approach combines two dimensions:

- **Confidence in the comparison itself** — how certain is the system that the documented claim and the extracted reality

actually refer to the same thing and were correctly extracted? (E.g., string-matching "the payments database" to a specific RDS instance is inherently less certain than matching a service name to a repo.)
- **Severity of the discrepancy** — how much does this drift matter if acted upon? A stale diagram label is low severity; a runbook pointing on-call to the wrong failover procedure during a database outage is high severity, regardless of how confident the comparison is.

Routing follows from severity, not confidence: low-severity drift is queued for batch reconciliation on the next pass; medium-severity drift gets an inline "unverified — needs review" flag in the artifact immediately (the artifact stays published, but visibly caveated); high-severity drift — anything touching security boundaries, data stores, or incident-response procedures — blocks the artifact from "trusted" status and opens a ticket for the owning team, following the same blast-radius-aware gating that Google's migration agents use for code changes.

Validation strategies, in order of cost

1. **Static analysis / structural comparison** (cheapest): does the dependency graph derived from code match the dependency graph implied by the architecture diagram?
2. **Runtime corroboration**: do traces/logs show traffic patterns consistent with the documented data flow? (Netflix's multi-source approach — combine network-level and application-level signals.)
3. **Cross-document corroboration**: does an independent source (a recent ADR, a postmortem) corroborate or contradict the claim?
4. **SME interview** (most expensive): only when the above are insufficient or the entity is high-severity, ask the owning team a specific, narrow question.

A realistic estimate of manual effort eliminated: structural and runtime corroboration (steps 1–2) can resolve the majority of "is this still accurate" checks for architecture and dependency documentation without any human involvement, because these are mechanical comparisons against data the org already collects. The SME interview step should be reserved for genuinely novel business-logic questions — the kind of thing that, today, can only be answered by "ask the person who wrote it."

8. Knowledge Packs & Outputs

Each pack is rendered from the knowledge graph, scoped by query (e.g., "everything related to Service A and its first-degree dependencies"). Below, each pack is mapped to the graph entities and agent outputs that produce it.

Architecture Knowledge Pack

Component	Derived from
System overview	Graph traversal: all services + teams + top-level dependencies, summarized by Documentation Agent
C4 diagrams (context/container/component)	Generated from the dependency graph (Repository + Architecture Agents); regenerated whenever drift detection finds a structural change
Service catalog	Direct export of Service/Team/Repo/API entities — essentially a Backstage-compatible view
Dependency graph	Direct graph export, cross-checked against runtime topology (Netflix-style)
Data flows	Derived from "reads/writes," "publishes/consumes" edges, annotated with confidence
Security boundaries	Derived from IaC (network policies, IAM) plus explicit annotations from ADRs; flagged for human review by default given the stakes of getting this wrong

Operations Knowledge Pack

Component	Derived from
Runbooks	Existing runbooks (verified against current architecture) plus new runbooks generated from postmortems (Section incident-to-knowledge)
Alerts	Linked from observability config; cross-referenced to the service/runbook they relate to
Escalation paths	Team ownership graph + on-call schedule integration
Common failures	Aggregated from incident history per-service, ranked by frequency/recency
Recovery procedures	Runbook entities, versioned, with provenance back to the postmortem that produced/updated them

Developer Knowledge Pack

Component	Derived from
Local setup	README + CI/CD config analysis; drift-checked against actual build steps
Build process	CI/CD pipeline definitions, summarized
Testing strategy	Test directory structure + coverage reports + CI config
Deployment process	CD pipeline + IaC, cross-checked against runtime (does the documented deploy target match where it actually runs?)
Coding standards	Linter/formatter configs + ADRs tagged as standards + tribal-knowledge FAQs distilled from PR review comments

AI Agent Knowledge Pack — the recursive/self-describing pack

This pack is structurally different from the other three: it describes *the knowledge system and the broader agent ecosystem to other agents*. It should be designed first among the four, even though it may be populated last, because its schema constrains how every other agent in the factory must describe its own actions (tools used, approvals required, escalation taken).

Component	Derived from / Notes
Available tools / MCP servers	Registry of MCP servers and tools available to agents in this org — itself a graph entity type, following the Backstage Soundcheck MCP precedent (Section 2)
Agent capabilities	Per-agent role description, scope, and known limitations (Section 6 roles, exposed as data)
Security boundaries	Which agents can read vs. write which entity types; which MCP tools require which scopes
Human approval gates	The Governance Agent's policy table (Section 10), exposed so other agents/humans can see what triggers review
Escalation policies	Who/what gets notified for disputed claims, high-severity drift, or failed validations

Why this matters now. The Spotify Backstage 2026 changelog already shows MCP actions allowing agents like Claude Code to query and manage compliance checks conversationally. An organization deploying this knowledge system will very likely have *other* agents (coding agents, SRE agents, support agents) that need to know what this system is, what it's allowed to do, and where its boundaries are. The AI Agent Knowledge Pack is that interface.

9. Design Decisions and Trade-offs (ADR-style)

ADR-1: Knowledge graph as single substrate vs. four independent pipelines

Decision: One knowledge graph; four packs are views/queries over it.

Alternatives considered: Four independent generation pipelines (one per pack), each with its own extraction logic.

Trade-off accepted: Higher upfront design cost for a unified schema (Section 5) and the risk that the schema becomes a bottleneck for new entity types. In exchange, the four packs cannot drift from each other, and new packs/views can be added later without new extraction pipelines.

ADR-2: Event-driven factory vs. scheduled batch regeneration

Decision: Agents are triggered by change events (commit, PR merge, incident close, ticket transition).

Alternatives considered: Nightly/weekly batch regeneration of all packs.

Trade-off accepted: Event-driven requires more infrastructure (event bus, idempotent agents, partial-update logic in the Documentation Agent) than "just re-run everything nightly." In exchange, "living" documents are actually live — a runbook can be flagged stale within minutes of the relevant code change, not up to a week later.

ADR-3: Confidence-and-severity routing vs. binary publish/reject

Decision: Drift findings and new facts are scored on confidence and severity independently and routed accordingly (Section 7), rather than a single "is this correct?" gate.

Alternatives considered: Binary gate — either an agent's output passes validation and publishes, or it's discarded/queued for full human review.

Trade-off accepted: More complex routing logic and more states for artifacts to be in (verified / unverified / disputed / superseded). In exchange, low-stakes updates (a renamed internal variable referenced in a doc) don't compete for the same human review queue as high-stakes updates (a security boundary changed) — this is essential for the system to scale without becoming a review bottleneck itself.

ADR-4: SME interviews as last resort vs. primary tribal-knowledge mechanism

Decision: The SME Interview Agent is invoked only when structural/runtime/cross-document validation (Section 7, strategies 1–3) is insufficient.

Alternatives considered: Proactively interview SMEs on a schedule to build out tribal-knowledge coverage.

Trade-off accepted: Slower initial coverage of "why" knowledge (decisions, rationale) that only humans currently hold. In exchange, the system avoids becoming a source of interruption fatigue — a well-known failure mode for any system that asks engineers questions, and the fastest way to get a tool ignored or actively avoided.

ADR-5: Additive to existing tools vs. new system of record

Decision: The system reads from and writes back into existing tools (PRs, wikis, runbook repos) where possible; the knowledge graph itself is a derived index, not a new place teams must go to update information.

Alternatives considered: A new unified knowledge portal that becomes the canonical place to edit documentation (a "new Confluence").

Trade-off accepted: Writing back into heterogeneous tools (a PR comment here, a wiki page edit there, a runbook commit elsewhere) is significantly harder than writing to one database. In exchange, adoption doesn't require a migration, and teams keep their existing review habits — directly following the Spotify lesson that ownership metadata living in the existing git workflow is what made it maintainable.

10. Governance, Trust, and Human-in-the-Loop

"Trusted knowledge artifact" needs an operational definition, or the term is marketing rather than a system property. This section proposes one, directly informed by the Netflix principle that wrong-but-confident information is the primary risk.

Trust levels

Level	Meaning	How an artifact gets here
Verified	Claim corroborated by structural analysis and/or runtime data, high confidence, no open disputes.	Automated — Validation Agent, no human gate required.
Unverified	Claim has not been contradicted, but also hasn't been independently corroborated (e.g., a new ADR's stated rationale).	Automated, visibly labeled in the artifact.
Disputed	Two or more sources disagree and validation could not resolve it.	Automated detection; surfaced to owning team, doesn't block other parts of the artifact from publishing.
Pending review	High-severity change (security boundary, data store, incident-response procedure) awaiting human sign-off.	Governance Agent policy, routed to owning team.
Approved / Trusted	Either auto-approved (high confidence + low severity) or explicitly signed off by a human reviewer.	Governance Agent, with full audit trail.
Superseded	Replaced by a newer verified claim; retained for history/provenance.	Automatic on new verified claim for the same entity/relationship.

What earns automatic ("auto-approvable") status

Following Google's confidence-bucket approach to migrations (Section 2), automatic approval should require *both* high confidence (corroborated by ≥2 independent sources, e.g., structural + runtime) *and* low blast radius (the affected entity is not tagged as security-sensitive, customer-facing-critical, or part of an active incident's blast radius). Anything failing either test routes to a human — specifically the owning team identified via the ownership graph (ADR-5's dependency on Section 5's "owns" edge).

Audit trail requirements

Every published claim must be traceable to: the source artifact(s), the extracting agent and model version, the validation evidence (what was compared against what), and — for anything that passed through governance — the approval decision and approver (human or policy rule ID). This is what allows the system to be debugged when it's wrong, and what allows trust to be calibrated over time (e.g., "auto-approvals from the Architecture Agent for dependency-graph updates have a 99.2% human-agreement rate over the last quarter — safe to keep on auto-approve").

11. Phased Roadmap

The roadmap sequences work so that each phase produces standalone value (avoiding a "big bang" launch) while building toward the full architecture.

Phase 0 — Foundations (graph schema + ingestion for one domain)

- Stand up the knowledge graph schema (Section 5) for a single pilot domain (e.g., one team's services).
- Build GitHub + CI/CD connectors and the Code Understanding Agent (Repository Agent) — establish the structural "ground truth" graph, the Kythe-equivalent.
- No documentation generation yet — the deliverable is "can the system accurately represent what exists and how it connects?"

Phase 1 — Drift Detection (the wedge, Section 7)

- Ingest existing documentation (architecture docs, runbooks, READMEs) for the pilot domain into the document store.
- Build the Drift Detection Agent and confidence/severity scoring model.
- Ship a standalone "drift report" — even without generating new documentation, surfacing "this doc says X, code says Y" is independently valuable and builds trust in the underlying comparisons.

Phase 2 — Validation, Governance, and the first Knowledge Pack

- Build the Validation and Governance Agents and the trust-level model (Section 10).
- Generate the Architecture Knowledge Pack for the pilot domain (highest-leverage pack — depends most directly on the structural graph already built in Phase 0).
- Establish the human review workflow (routing to owning teams) — this is where most of the org-change-management work happens, and doing it on one pack first de-risks the rollout.

Phase 3 — Tribal Knowledge & Incident-to-Knowledge

- Add Slack/Teams, meeting transcript, and incident/postmortem connectors plus the NL Extraction and SME Interview Agents.
- Generate the Operations Knowledge Pack, with incident-to-knowledge automation as the primary new capability (Section "Incident-to-Knowledge Automation" in the original brief).
- Calibrate auto-approval thresholds using Phase 2's audit data.

Phase 4 — Developer Pack, AI Agent Pack, and Multi-Domain Scale-Out

- Generate the Developer Knowledge Pack (lower risk — mostly derived from existing structured sources).
- Build the AI Agent Knowledge Pack and expose the knowledge graph via MCP (Section 8) so other internal agents can consume it.
- Scale connectors and agents from the pilot domain to additional teams/domains, using the calibrated trust model from Phase 3.

12. Risks, Open Questions, and Closing Notes

Risks

Risk	Mitigation
Agents produce confident-but-wrong architecture claims that get embedded into onboarding/on-call materials	Confidence/severity routing (ADR-3), trust levels (Section 10), and explicit "disputed" status rather than silent override of conflicting sources
SME interview fatigue causes teams to ignore or actively avoid the system	SME interviews as last resort (ADR-4); track interview frequency per team as a health metric
Knowledge graph schema becomes a bottleneck as new entity types are needed	Design the schema with an extensible "annotation"/custom-property mechanism from Phase 0, informed by how Backstage's catalog model handles arbitrary metadata
Write-back to heterogeneous tools (wikis, runbook repos) is brittle across tool versions/permissions	Start read-only (Phases 0–1 produce reports, not edits); introduce write-back gradually per-tool, gated by governance
Security-sensitive information (data flows, access boundaries) is surfaced too broadly by the AI Agent Knowledge Pack	Treat security boundary entities as pending-review by default (ADR/Section 8) and apply the same access controls to the knowledge graph that apply to the underlying systems

Open questions for the organization adopting this

- What is the org's tolerance for auto-published documentation changes, even at "verified, low blast radius" trust level? Some regulated environments may require human sign-off on everything regardless of confidence.
- How will the system handle organizational knowledge that is intentionally *not* written down for sensitivity reasons (e.g., a known-fragile system nobody wants formally documented as "do not touch")?
- Who owns the knowledge graph schema itself as a piece of infrastructure — is it a platform team responsibility, analogous to who owns Backstage's catalog model at Spotify?

Closing

The case studies in Section 2 show that the building blocks of this system already exist, separately, inside major tech companies — Spotify's catalog solved discovery and ownership, Google's Kythe solved structural code understanding at scale, and Netflix's Service Topology solved multi-source reconciliation for dependency truth. What none of these systems originally had was a documentation layer that wrote itself back, continuously, with calibrated trust. The architecture in this brief is, in essence, the proposal that LLM agents are now capable enough to be that missing layer — provided the surrounding system is designed, from the start, around the same lessons these companies learned the hard way: ownership before documentation, structure before narrative, multiple sources before any single source, and confidence as a first-class, visible property of every claim.